

World Cup Predictor 2026

Durée	4h (sur site) ou 1 semaine (à distance)
Stack	Libre — à justifier dans le README
Livraison	Repo GitHub (lien envoyé par email)
Notation	/20 (Must-have) + /10 (Bonus)

Contexte

La Coupe du Monde 2026 (Canada / Mexique / États-Unis, du 11 juin au 19 juillet 2026) est la première édition à 48 équipes. Le tournoi se déroule en deux phases :

- **Phase de groupes** : 12 groupes de 4 équipes, chaque équipe joue 3 matchs.
- **Phase à élimination directe** : 32 équipes qualifiées (les 2 premiers de chaque groupe + les 8 meilleurs troisièmes), réparties en 5 tours (16èmes de finale, 8èmes de finale, quarts de finale, demi-finales, finale).

Vous devez développer une application web de **pronostics** permettant à un utilisateur de prédire le vainqueur de chaque match, du premier match de poule à la finale. Le bracket se met à jour dynamiquement au fil des choix, et l'utilisateur découvre son champion du monde.

Citation à intégrer au README

"Une application qui permet à n'importe quel fan de football de bâtir son scénario complet pour la Coupe du Monde 2026, du premier coup d'envoi à la finale, et de le sauvegarder."

Données fournies

Un fichier `teams-2026.json` est fourni avec :

- Les **48 équipes qualifiées** avec nom, code drapeau ISO, confédération, classement FIFA et groupe.
- La **composition des 12 groupes** (A à L), issue du tirage officiel du 5 décembre 2025.
- Les **règles de format** et de départage du tournoi.

Les drapeaux sont accessibles via l'URL :

```
https://flagcdn.com/w160/{code}.png
```

Par exemple `https://flagcdn.com/w160/fr.png` pour la France. Les codes spéciaux `gb-eng` et `gb-sct` sont utilisés pour l'Angleterre et l'Écosse.

Fonctionnalités obligatoires (Must-have)

Ces fonctionnalités sont notées sur 20 points et constituent le socle minimum attendu.

#	Critère	Points
1	Chargement et parsing du JSON fourni (48 équipes, 12 groupes)	1
2	Affichage de la phase de groupes (12 groupes de 4 équipes avec drapeaux)	3
3	Saisie du vainqueur ou du score de chaque match de poule	3
4	Calcul automatique du classement de chaque groupe (points, diff. de buts, buts marqués)	3
5	Sélection automatique des 8 meilleurs 3èmes pour les 16èmes de finale	2
6	Génération du tableau à élimination directe (32 → 16 → 8 → 4 → 2 → 1)	3
7	Propagation du vainqueur au tour suivant à chaque choix	2
8	Persistance des pronostics (la sélection survit au refresh)	1
9	Responsive mobile + desktop	1
10	Qualité du code (structure, nommage, lisibilité, pas de code mort)	1

Fonctionnalités bonus

Notées sur 10 points supplémentaires. Aucune n'est obligatoire, mais elles valorisent fortement la copie. Privilégier la qualité d'implémentation à la quantité.

Fonctionnalité	Points
Animation lors de la propagation du vainqueur dans le bracket	1
Mise en évidence du parcours complet d'une équipe (clic = highlight)	2
Mode score (buts marqués) au lieu de simple vainqueur, avec prolongations / tirs au but	2
Export du bracket complet en image PNG (partage réseaux sociaux)	2
Statistiques de fin (équipe favorite, confédération dominante, etc.)	1
Comparaison avec un bracket de référence (basé sur le classement FIFA)	2
Tests unitaires sur la logique métier (classement, sélection des 3èmes)	1
Mode sombre	1

Règles métier à respecter

Classement d'un groupe

Les équipes d'un groupe sont classées dans l'ordre suivant :

- Nombre de points (victoire = 3, nul = 1, défaite = 0)
- Différence de buts globale
- Buts marqués globalement
- Confrontation directe entre équipes à égalité (points → diff. de buts → buts marqués)
- Score Fair-Play (à simuler ou ignorer dans le cadre de l'examen)
- Classement FIFA (fourni dans le JSON)

Sélection des 8 meilleurs troisièmes

Sur les 12 troisièmes de groupe, seuls les 8 meilleurs se qualifient pour le Round of 32. Ils sont classés selon les mêmes critères que ci-dessus, mais à l'échelle inter-groupes. **C'est l'algorithme le plus exigeant de l'examen.**

Format des 16èmes de finale

32 équipes s'affrontent en 16 matchs. La répartition exacte des matchs (qui rencontre qui selon les positions de groupe) suit un schéma défini par la FIFA. Le candidat peut soit implémenter le schéma officiel, soit en proposer un personnel à condition de le documenter clairement dans le README. Les deux approches sont valorisées également.

Suite de la phase à élimination directe

Après les 16èmes de finale : 8èmes de finale (16 → 8), quarts de finale (8 → 4), demi-finales (4 → 2), finale (2 → 1). Pas de match pour la 3ème place exigé, mais bonus si implémenté.

Livrables attendus

- **Repo GitHub** (public ou privé avec accès donné au correcteur) avec un historique de commits cohérent (pas un unique commit final).
- **README.md** contenant : instructions d'installation et de lancement, stack technique choisie et justification, fonctionnalités implémentées (must-have et bonus), difficultés rencontrées et compromis assumés, captures d'écran (optionnel mais apprécié).
- **Code source** propre, structuré et lisible.
- **.gitignore** correct (pas de `node_modules`, `.env`, `builds`, etc.).

Critères transversaux observés

Au-delà du barème détaillé, le correcteur évaluera également :

- Cohérence et granularité des commits Git (messages clairs, étapes logiques).
- Justification des choix techniques (lib, gestion d'état, structure des composants).
- Gestion adaptée de l'état (pas de Redux si `useState` suffit, pas de tout-en-prop-drilling si Context aide).
- Gestion des cas limites : score invalide, scores partiels, équipes à égalité parfaite, refresh en plein parcours.
- Accessibilité de base (attributs `alt` sur les drapeaux, contraste suffisant, navigation clavier).
- Absence de `console.log`, de code mort, de dépendances inutiles.
- Performance acceptable même avec un bracket complet rempli.

Anti-patterns pénalisés

- Données en dur dans le code au lieu d'utiliser le JSON fourni.
- Un seul commit « initial commit » avec tout le code.
- README absent, minimaliste ou en anglais incompréhensible.
- Recalcul complet du bracket à chaque clic au lieu d'une mise à jour ciblée.
- Copier-coller manifeste depuis un tutoriel sans compréhension (questions orales possibles).
- Aucune gestion d'erreur sur le parsing ou les saisies utilisateur.
- Variables non typées si TypeScript est annoncé dans le README.

Conseils pour bien réussir

- **Commencer par modéliser les données** sur papier ou dans un fichier `types.ts` avant d'écrire la moindre ligne d'UI.
- **Implémenter le calcul de classement et la sélection des 8 meilleurs 3èmes en premier**, idéalement testés unitairement, avant de toucher à l'UI. C'est le cœur métier.
- **Décomposer en composants réutilisables** : un composant de match doit fonctionner aussi bien en poule qu'en phase finale.
- **Ne pas surinvestir le visuel** au détriment des fonctionnalités. Une UI propre vaut mieux qu'une UI flashy à moitié fonctionnelle.
- **Commits fréquents** et messages explicites (« feat: classement des groupes » plutôt que « update »).
- **Lire le JSON en entier au démarrage** avant de coder, pour comprendre la structure de données disponible.

Questions et support

Toute question fonctionnelle peut être posée par email au début de l'examen. Une fois l'examen commencé, les choix d'implémentation et d'interprétation des règles sont laissés au candidat — ils doivent être documentés dans le README.

Bon courage et amusez-vous bien !